

MiniKV Server

Introducción

La arquitectura [cliente-servidor](#) es la predominante en la internet debido a su capacidad para centralizar recursos, mejorar la seguridad y gestionar datos de forma eficiente. Permite que múltiples usuarios (clientes) accedan simultáneamente a servicios, bases de datos o aplicaciones compartidas alojadas en una máquina central (servidor), facilitando el mantenimiento, la escalabilidad y la organización de la información.

En este ejercicio se propone implementar un **servidor de base de datos** en Rust llamado `minikv-server`. El sistema permitirá a **múltiples clientes** operar sobre la misma base de datos minikv desarrollada previamente. La comunicación se realizará mediante [sockets](#). Desde el lado del servidor, cada conexión se procesará en un [thread](#) distinto.

Para más información sobre concurrencia en Rust, referirse a [The Rust Programming Language - Capítulo 16](#). Como ejemplo de un proyecto que utiliza concurrencia y redes, pueden leer [The Rust Programming Language - Capítulo 21](#).

Binarios Entregables

El proyecto de Rust deberá estar compuesto por dos binarios:

- `minikv-server`
- `minikv-client`

Para más información sobre cómo estructurar el proyecto, referirse [The Rust Programming Language - Capítulo 7](#).

Para validar que el proyecto está bien estructurado, ejecutar `cargo build` deberá generar dos binarios, uno con el nombre de `minikv-server`, y otro con el nombre de `minikv-client`.

Servidor

El servidor recibirá como argumento la dirección a través de la cual escuchará conexiones entrantes de los operadores.

```
cargo run --bin minikv-server -- 192.168.0.0:12345
```

Al iniciar, el servidor tiene que leer el estado de `.minikv.data` y el `.minikv.log` para construir la base de datos en memoria. Se debe persistir el estado a medida que se ejecuten los comandos.

Por cada conexión entrante, el servidor debe crear un hilo nuevo para manejar esa conexión.

Cliente

El cliente recibirá como argumento la dirección del servidor, y leerá las operaciones a enviar al servidor a través de STDIN.

```
cargo run --bin minikv-client -- 192.168.0.0:12345
```

El cliente deberá establecer una conexión con el servidor, y enviar las operaciones secuencialmente. No se debe validar que las operaciones sean válidas, eso es trabajo del servidor. El resultado de las operaciones deberá imprimirse por STDOUT.

Ejemplo:

```
stdin : set a b
stdout : OK
stdin : get a
stdout : b
stdin : set c d
stdout : OK
stdin : snapshot
stdout : OK
```

Deben existir timeouts para que el programa no quede colgado esperando al server, y estos deben ser constantes configurables.

Errores

Los casos de error se dividen en 3 categorías:

- Error de cliente
- Error de comunicacion
- Error del servidor

Los errores se imprimirán por STDOUT, y deberán respetar el siguiente formato.

```
ERROR "<motivo>"
```

Errores de cliente

Estos errores son *recuperables*, es decir que puede continuar la ejecución del programa servidor y la comunicación entre cliente y servidor. Cuando el servidor los detecta se debe enviar el error al cliente en el formato especificado, pero manteniendo la conexión activa.

Los códigos de error son:

- NOT FOUND
- EXTRA ARGUMENT
- MISSING ARGUMENT
- UNKNOWN COMMAND

Errores de comunicacion

Estos implican el cierre de la comunicación con el cliente pero no afectan la ejecución del programa servidor. Como no puede comunicarse el error con el cliente se deberá manejar por separado en ambos programas:

- En el server se debe imprimir el error en el formato especificado, y finalizar ese hilo, pero **NO** el servidor completo.
- En el cliente se debe imprimir el error, y finalizar la ejecución.

Los códigos de error son:

- TIMEOUT : El server tarda demasiado en contestar, lo cual puede indicar que está caído.
- CONNECTION CLOSED : La conexión se cierra de forma repentina.
- CLIENT SOCKET BINDING : El cliente no puede bindear un socket en la dirección especificada del server.

Errores del server

Estos errores son irrecuperables e implican la finalización del programa servidor

Los códigos de error son:

- INVALID ARGS : No se reciben los argumentos esperados en la ejecución del server.
- SERVER SOCKET BINDING : El servidor no puede bindear un socket en la dirección especificada.
- INVALID DATA FILE
- INVALID LOG FILE

Operaciones

Las operaciones que admite el servidor son las siguientes:

- `set`
- `get`
- `length`
- `snapshot`

Los comandos siguen la misma lógica que en la entrega anterior.

Protocolo de Comunicación

El [protocolo de comunicación](#) será sencillo. El cliente y servidor intercambiarán mensajes de texto. Para más información, ver [text-based protocols](#).

El protocolo permitirá enviar los comandos definidos. Al recibir el mensaje el servidor ejecutará el comando y responderá con el output del comando.

Ejemplo 1

Consideramos un único servidor, y un único cliente.

```
client : set a b
server : OK
client : get a
server : b
client : set c d
server : OK
client : snapshot
server : OK
```

Ejemplo 2

En caso de una operación inválida, respondemos con un mensaje de error e ignoramos la operación.

```
client : set a b
server : OK
client : set a b c
server : ERROR "EXTRA ARGUMENT"
client : get a
server : b
```

Ejemplo 3

En el caso de múltiples clientes vamos a poder tener operaciones intercaladas, por ejemplo

```
client1 : set a b
server  : OK
client2 : set a c
server  : OK
client1 : get a
server  : c
```

Restricciones

- No se permite que el programa lance un `panic!()`. Es decir, no se puede utilizar `.unwrap()` o `.expect()`. Todo caso de error deberá manejarse idiomáticamente con las estructuras y funciones brindadas por el lenguaje.
- No se permite utilizar la función `exit()`. Se deberá salir del programa finalizando el scope de la función `main`.
- No se permite utilizar el módulo `mem` para la manipulación de memoria.
- Para realizar un uso adecuado de memoria y respetar las reglas de *ownership* se deberá evitar el uso de `.clone()` y `.copy()` en las estructuras principales de datos.

Requerimientos no funcionales

Los siguientes son los requerimientos no funcionales para la resolución del proyecto:

- El proyecto deberá ser desarrollado en la última versión estable de Rust (1.94), usando las herramientas de la biblioteca estándar.
- Se deben implementar tests unitarios.
- No se permite utilizar *crates* externos.
- El código fuente debe compilarse en la versión estable del compilador.
- No se permite utilizar bloques `unsafe`.
- El código deberá funcionar en ambiente Unix / Linux.
- Los programas deberán ejecutarse en la línea de comandos.
- La compilación no debe arrojar `warnings` del compilador, ni del *linter* `clippy`.
- Las funciones y los tipos de datos (`struct`, `enum`) deben estar documentados siguiendo el [estándar de cargo doc](#).
- El código debe formatearse utilizando `cargo fmt`.
- Las funciones no deben tener una extensión mayor a 30 líneas. Si se requiriera una extensión mayor, se deberá particionarla en varias funciones.
- Cada tipo de dato implementado debe ser colocado en un módulo (archivo) independiente.

- No debe existir un **busy loop**.

El proyecto deberá realizarse de manera individual. Cualquier tipo de copia significa la expulsión automática de la materia. No está permitido el uso de código generado por ninguna IA, ni copiar código de soluciones existentes en Internet.

Entrega

Fecha de entrega: 09/04 - 18:00

La entrega se realizará por Algotron. Para que la entrega se considere válida, deberán pasar todas las verificaciones de la plataforma. En caso contrario, no se podrá continuar en la materia.